1 Sorğu serverdə necə işləyir

Kafedə müştəri telefondan menyunu açır. Ekranda məhsulların görünməsi üçün telefon serverdən siyahı istəyir. **Server** burada məlumatı hazırlayıb cavab verən proqramdır.

Bu dərsdə yalnız bir hadisəni izləyəcəyik: müştəri menyunu istəyir, server menyunu qaytarır. Həmin hadisənin başlanğıcdan sona qədər keçdiyi yol onun **həyat dövrüdür**. İngiliscə buna **lifecycle** deyilir.

Müştəri ilə server arasındakı yol

```
Telefon: "Menyunu ver."
             |
             v
Server: menyunu tapir ve cavab hazirlayir.
             |
             v
Telefon: gelen menyunu ekranda gosterir.
```

Telefonun göndərdiyi istək **request**, serverin qaytardığı cavab **response** adlanır. “Sorğu” və “cavab” deyəndə bunları nəzərdə tutacağıq.

Bu mərhələdə yadda saxla: server bir sorğunu qəbul edir, onun üçün iş görür və cavab qaytarır. Lifecycle həmin işin yolunu təsvir edir.

Sorğunun içində nə gəlir

Telefon serverə sadəcə “menyu” sözünü göndərmir. Sorğunun hansı işi istədiyini və hansı ünvana getdiyini bildirir. Məsələn:

HTTP sorğusunun sadələşdirilmiş görünüşü

```
GET /menu
```

HTTP müştəri ilə serverin sorğu və cavabı hansı qayda ilə göndərdiyini müəyyən edir. Yuxarıdakı `GET` məlumat oxumaq istədiyimizi bildirir. `/menu` hansı məlumatı istədiyimizi göstərən yoldur.

Serverdə `/menu` üçün yazılmış kod seçilir. Ünvanla onu işləyəcək kodun uyğunlaşdırılmasına **routing** deyilir. Seçilən işləyici nöqtəyə isə **endpoint** deyilir.

Server menyunu tapanda `200` statusu ilə cavab verə bilər. **Status kodu** cavabın nəticəsini bildirən rəqəmdir. Menyu tapılmırsa `404`, gözlənilməz server xətası varsa `500` ola bilər.

Beləliklə, ardıcıllıq budur: **ünvan gəlir, həmin ünvanın kodu seçilir, kod cavab hazırlayır.**

İlk oxunuşda yadda saxlamalı əsas əlaqə budur: `GET /menu` sorğudur; həmin sorğuya cavab verən kod endpoint-dir.

Sorğu niyə birbaşa menyu koduna getmir

Kafedə bəzi əməliyyatlar yalnız daxil olmuş istifadəçilər üçündür. Məsələn, “mənim sifarişlərim” ekranını açmazdan əvvəl istifadəçini tanımaq lazımdır.

Server buna görə əsas işi görməzdən əvvəl bir neçə yoxlamadan keçir. Bu yoxlamaları aparan komponentlərə **middleware** deyilir. Onların ardıcıllığı **pipeline**, yəni emal zənciridir.

Bir sorğunun nümunə yolu

```
Sorqu gelir
    |+    v
Xeta yaranarsa onu tutacaq qat
    |
    v
Istifadecinin kim oldugunun yoxlanmasi
    |
    v
Istifadecinin bu ise icazesinin yoxlanmasi
    |
    v
Sifarisleri getiren esas kod
```

Kimliyi müəyyən etməyə **authentication** deyilir: “Bu adam kimdir?” İcazəni yoxlamağa **authorization** deyilir: “Bu adam bu işi görə bilərmi?” Daxil olmuş müştərinin başqa müştərinin sifarişinə baxmağa icazəsi olmaya bilər.

Middleware sorğunu növbəti mərhələyə ötürə və ya özü cavab verib dayandıra bilər. İcazə yoxdursa əsas sifariş koduna getmədən cavab qaytarılır.

Nəticə: middleware əsas işə çatana qədər və iş bitəndən sonra görüləcək ortaq işləri təşkil edir.

Əvvəl və sonra kodunun sırası

İki middleware düşün. Birincinin adı A, ikincinin adı B-dir. Hər ikisi əvvəl bir qeyd yazır, növbəti mərhələni çağırır, sonra yenə qeyd yazır.

İşləmə sırası

```
A: evvel
    B: evvel
        Menyunu hazirlayan kod
    B: sonra
A: sonra
```

Sıra niyə belədir? A, B-ni çağırıb onun bitməsini gözləyir. B əsas kodu çağırıb onun bitməsini gözləyir. Əsas kod bitəndə əvvəl B-nin, sonra A-nın qalan hissəsi işləyir.

ASP.NET Core-da növbəti mərhələni təmsil edən parametr çox vaxt `next` adlanır. **ASP.NET Core** C# ilə veb server proqramı qurmaq üçün istifadə edilən çərçivədir. Çərçivə hazır server və sorğu emalı mexanizmlərini təqdim edir.

Əgər B növbəti mərhələni çağırmasa, əsas kod işləməz. Əgər əsas kod xəta ilə dayansa, “sonra” hissələrinin hamısının adi qaydada işləyəcəyinə güvənmək olmaz. Xətadan asılı olmayaraq görülməli təmizlik işi üçün `finally` bloku istifadə edilir. Bu blokun işi koddan çıxarkən təmizliyi yerinə yetirməkdir.

Yadda saxla: sorğu zəncirin içinə doğru gedir, uğurlu iş bitəndə davam kodu əks ardıcıllıqla tamamlanır.

Sorğu bitəndə nə bitir

Telefon menyunu aldı. Bu sorğu üçün iş bitdi. Amma server proqramı bağlanmır: növbəti müştərinin sorğusunu qəbul etməyə davam edir.

Burada iki müxtəlif ömür var:

Nəyin ömrü	Nə vaxt bitir
Bir HTTP sorğusu	Həmin sorğunun emalı tamamlananda.
Server proqramı	Tətbiq dayandırılanda və ya proses bağlananda.

Proses əməliyyat sistemində işləyən proqramın mühitidir. Eyni server proqramı çoxlu sorğunu emal edə bilər.

Sorğu üçün yaradılmış köməkçi obyektlərin də öz ömrü olur. Bəzisi yalnız bu sorğu üçün, bəzisi isə bütün server üçün paylaşılır. Bunun necə seçildiyini növbəti DI dərsində görəcəyik.

Müştəri əlaqəni kəsəndə server bunu ləğv siqnalı ilə bildirə bilər. Amma başladılmış hər iş özbaşına dərhal yox olmur. İşi görən kod siqnalı nəzərə almalıdır. Bunu async dərsində ayrıca açacağıq.

Özünü yoxla

Telefon eyni menyu ekranını iki dəfə yeniləsə nə olur? Adətən iki ayrı HTTP sorğusu göndərilir. Server proqramı isə həmin iki sorğu arasında işləməyə davam edir.

İcazə yoxlaması əsas kodu çağırmadan cavab verə bilərmi? Bəli. Middleware zənciri orada dayandıra bilər.